

Task2: Hazelcast

Tags

Виконала: Іванченко Марія

перевірити GitHub: <https://github.com/mariiaivanchenko/Software-Architecture-Task1/tree/hazelcast>

Встановити і налаштувати Hazelcast

Для цього перейшла за [посиланням](#) та скачала відповідний архів.

Сконфігурувати і запустити 3 ноди (інстанси) об'єднані в кластер або як частину Java-застосування, або як окремі застосування

Спершу я змінила назву кластеру, для цього поміняла/додала відповідні поля у файлах *hazelcast.xml* та *hazelcast-client.xml*.

hazelcast.xml

до:

```
<hazelcast xmlns="http://www.hazelcast.com/schema/config"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.hazelcast.com/schema/config
    http://www.hazelcast.com/schema/config/hazelcast-config-5.5.xsd">

  <!--
    The name of the cluster. All members of a single cluster must have the same cluster name
    configured and a client connecting to this cluster must use it as well.
  -->
  <cluster-name>dev</cluster-name>
```

після:

```
<hazelcast xmlns="http://www.hazelcast.com/schema/config"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.hazelcast.com/schema/config
    http://www.hazelcast.com/schema/config/hazelcast-config-5.5.xsd">

  <!--
    The name of the cluster. All members of a single cluster must have the same cluster name
    configured and a client connecting to this cluster must use it as well.
  -->
  <cluster-name>hello-world</cluster-name>
```

hazelcast-client.xml

до:

```
<hazelcast-client xmlns="http://www.hazelcast.com/schema/client-config"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.hazelcast.com/schema/client-config
    http://www.hazelcast.com/schema/client-config/hazelcast-client-config-5.5.xsd">

  <network>
```

після:

```
<hazelcast-client xmlns="http://www.hazelcast.com/schema/client-config"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.hazelcast.com/schema/client-config
    http://www.hazelcast.com/schema/client-config/hazelcast-client-config-5.5.xsd">

  <cluster-name>hello-world</cluster-name>
  <network>
```

Потім за допомогою команди запустила кластер з трьома елементами.

```
hazelcast-5.5.0/bin/hz start
```

```
2025-02-25 18:16:25,746 [ INFO] [main] [c.h.s.logo]: [127.0.0.1]:5703 [hello-world] [5.5.0]
  o   o   o   o   o   o   o   o   o   o   o   o   o   o   o   o   o   o   o   o   o   o
  |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
  o---o o---o o---o o---o o---o o---o o---o o---o o---o o---o o---o o---o o---o o---o
  |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
  o   o   *   o   o   o   o   o   o   o   o   *   o   o   o   o   o   o   o   o   o   o
  |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
  o   o   *   o   o---o o---o o---o o---o *   o   o---o o---o o---o o---o o---o o---o

2025-02-25 18:16:25,746 [ INFO] [main] [c.h.system]: [127.0.0.1]:5703 [hello-world] [5.5.0] Copyright (c) 2008-2024, Hazelcast,
Inc. All Rights Reserved.
2025-02-25 18:16:25,753 [ INFO] [main] [c.h.system]: [127.0.0.1]:5703 [hello-world] [5.5.0] Hazelcast Platform 5.5.0 (20240726)
starting at [127.0.0.1]:5703
2025-02-25 18:16:25,754 [ INFO] [main] [c.h.system]: [127.0.0.1]:5703 [hello-world] [5.5.0] Cluster name: hello-world

2025-02-25 18:16:28,866 [ INFO] [hz.hopeful_perlman.IO.thread-in-0] [c.h.i.s.t.TcpServerConnection]: [127.0.0.1]:5703 [hello-wor
ld] [5.5.0] Initialized new cluster connection between /127.0.0.1:54364 and /127.0.0.1:5701
2025-02-25 18:16:28,865 [ INFO] [hz.hopeful_perlman.IO.thread-in-1] [c.h.i.s.t.TcpServerConnection]: [127.0.0.1]:5703 [hello-wor
ld] [5.5.0] Initialized new cluster connection between /127.0.0.1:54365 and /127.0.0.1:5702
2025-02-25 18:16:28,980 [ INFO] [hz.hopeful_perlman.generic-operation.thread-1] [c.h.i.c.ClusterService]: [127.0.0.1]:5703 [hell
o-world] [5.5.0]

Members {size:3, ver:3} [
  Member [127.0.0.1]:5701 - e1f0b84e-fd05-4e56-865b-06f49bd332c7
  Member [127.0.0.1]:5702 - d7e889ed-6a88-4ee2-9be5-e82c8e383754
  Member [127.0.0.1]:5703 - e6ef3d04-2838-444f-a10d-b619a752216f this
]
```

Другою командою перевірила, щоб вони працюють коректно.

```
➔ ~ hazelcast-5.5.0/bin/hz-cli --config hazelcast-5.5.0/config/hazelcast-client.xml cluster
State: ACTIVE
Version: 5.5.0
Size: 3

ADDRESS                UUID
[127.0.0.1]:5701       e1f0b84e-fd05-4e56-865b-06f49bd332c7
[127.0.0.1]:5702       d7e889ed-6a88-4ee2-9be5-e82c8e383754
[127.0.0.1]:5703       e6ef3d04-2838-444f-a10d-b619a752216f
➔ ~
```

Продемонструйте роботу Distributed Map

Спершу створюю віртуальне середовище та необхідну скачую бібліотеку.

```
python3 -m venv /Users/mariia/Desktop/Task2/.venv
source /Users/mariia/Desktop/Task2/.venv/bin/activate
pip install hazelcast-python-client
```

```

→ Task2 python3 -m venv /Users/mariia/Desktop/Task2/.venv
source /Users/mariia/Desktop/Task2/.venv/bin/activate
(.venv) → Task2 pip install hazelcast-python-client
Requirement already satisfied: hazelcast-python-client in ./venv/lib/python3.13
/site-packages (5.5.0)

[notice] A new release of pip is available: 25.0 -> 25.0.1
[notice] To update, run: pip install --upgrade pip
(.venv) → Task2 █

```

За допомогою наступного коду, заливаю дані на кластер.

```

import hazelcast
import uuid

if __name__ == "__main__":
    client = hazelcast.HazelcastClient(
        cluster_name="hello-world",
    )

    map = client.get_map("distributed-map").blocking()

    # write data
    for i in range(1000):
        map.put(i, uuid.uuid4())

    # read data
    for key, value in map.entry_set():
        print(key, value)

    client.shutdown()

```

Активація manager-center за наступною командою *management-center/bin/start.sh* та перевірка розподілу елементів між елементами кластеру.

hello-world

i
State: **ACTIVE**

Safe Members: **3/3**

Clients: **0**

Stream Processing and SQL enabled

VIEW CLUSTER



Map Statistics (In-Memory Format: BINARY)

RESET TIME

1 minute ago → now

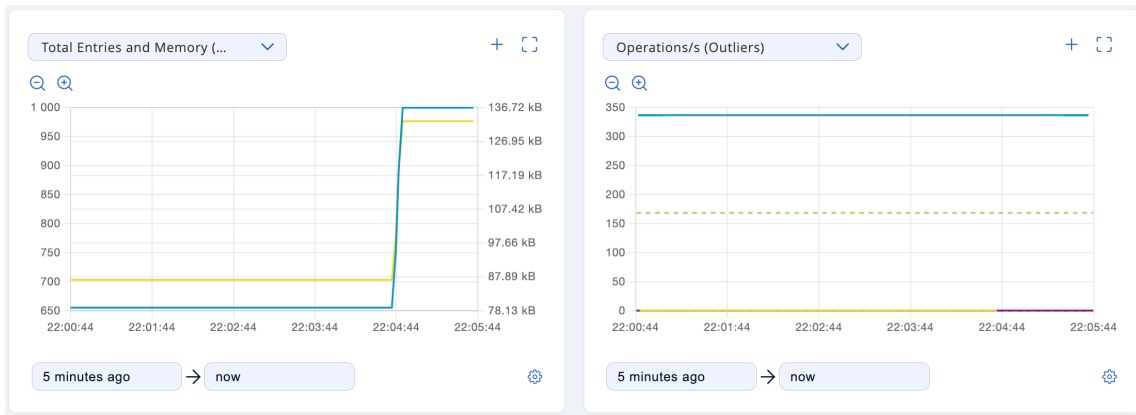
Default View

Member ^	Entries ^	Gets ^	Puts ^	Removals ^	Sets ^	Entry Memory ^
127.0.0.1:5701	337	0	337	0	0	44.76 kB
127.0.0.1:5702	319	0	319	0	0	42.37 kB
127.0.0.1:5703	344	0	344	0	0	45.69 kB
TOTAL	1 000	0	1 000	0	0	132.81 kB

1 - 3 of 3
Rows
10

Потестувала наступні випадки та подивилась як змінився розподіл даних по елементах(нодах):

- якщо відключити одну ноду



Map Statistics (In-Memory Format: BINARY)

RESET TIME

1 minute ago

→

now

Default View

Member

Entries

Gets

Puts

Removals

Sets

Entry Memory

127.0.0.1:5701	504	0	337	0	0	66.94 kB
127.0.0.1:5703	496	0	0	0	0	65.88 kB
TOTAL	1 000	0	337	0	0	132.81 kB

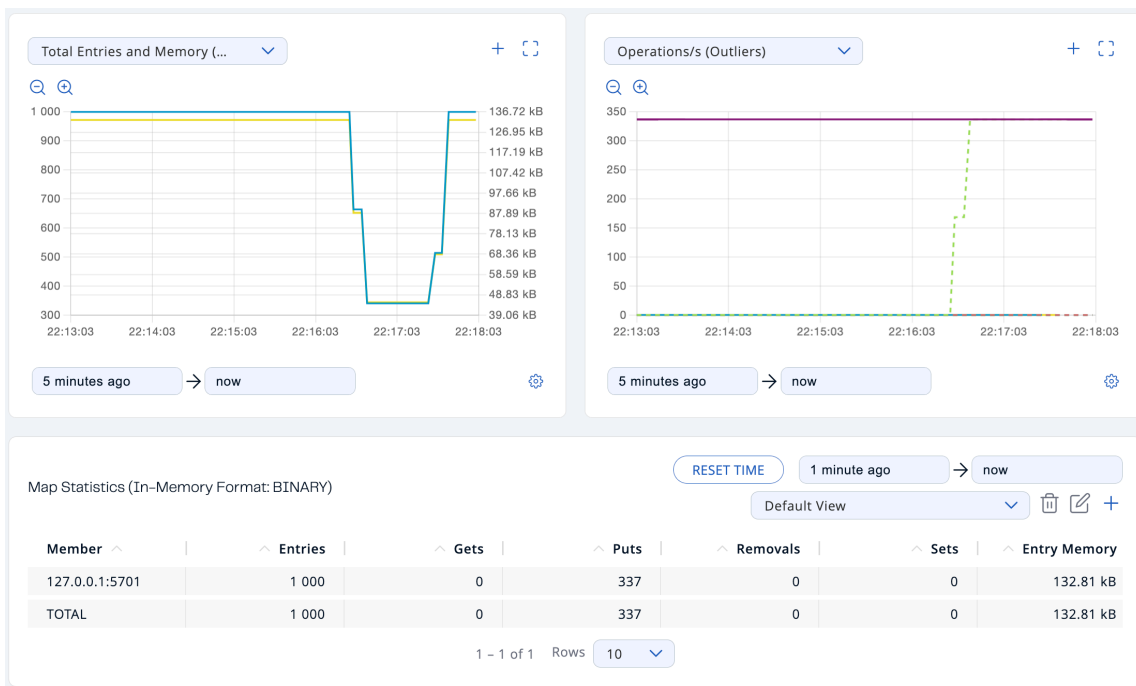
1 - 2 of 2

Rows

10

Після відключення однієї ноди можна помітити те, що стався перерозподіл даних між рештою нод, втрата даних не відбулась.

- відключити послідовно дві ноди



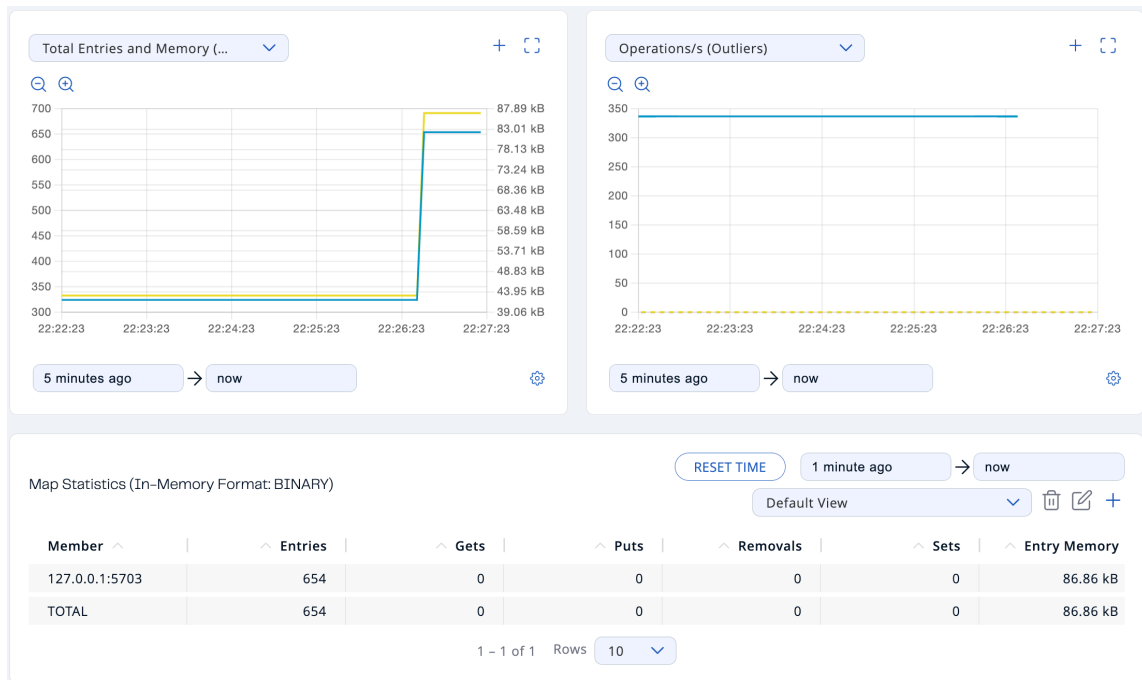
Після відключення двох нод послідовно можна помітити відповідні стрибки на графіках, як поступово впала кількість даних і поступово перерозподілилась та повернулась до початкового стану, втрата даних не відбулась.

- відключити одночасно дві ноди (емулюючи "падіння" серверів, чи використовуючи команду kill -9) (результати мають бути у протоколі)

```

~ lsof -n -i :5701 | grep LISTEN
java 3714 mariia 45u IPv6 0x4c83c6aa20287650 0t0 TCP 127.0.0.1:5701 (LISTEN)
~ lsof -n -i :5702 | grep LISTEN
java 39381 mariia 45u IPv6 0x4a32698400065294 0t0 TCP 127.0.0.1:5702 (LISTEN)
~ kill -9 3714 39381

```



Якщо емулювати падіння двох нод, можна помітити втрату даних. Це може бути спричинено тим, що копії відповідних даних на нодах були збережені на тій ж ноді і коли її примусово та спонтанно завершили, вони не були правильно відновлені. Але якщо копії даних були збережені на інших нодах, їх можливо відновити без втрати даних.

- Чи буде втрата даних?
Втрати даних не відбулось при відключенні одної, та двох нод

послідовно, але втрата даних відбулась коли ноди емулювали падіння, що може бути спричинено не коретним збереженням копії даних.

- Яким чином зробити щоб не було втрати даних?

На мою думку, можна зробити кілька копії даних, але ці копії зберігати на інших нодах серверу, якби у разі падіння поточної ноди її копії могли бути відновлені з інших нод кластеру. Однак якщо впаде весь кластер одночасно це не дозволить збереги дані, тому треба зберігати копії також на диску.

Продемонструйте роботу **Distributed Map without locks**

У цьому прикладі я створила 3 потоки, де кожен з них намагається отримати доступ до того ж самого елемента та збільшити його на одиницю. Оскільки доступ до одного і того елемента з різних потоків не є захищеним це викликає data race і кінцеве значення не буде дорівнювати 30.000, бо за кожної ітерації програми потоки зможуть доступитись до елемента різну кількість разів, невідомо в якій послідовності та скільки разів кожен з потоків буде збільшувати значення.

```
import hazelcast
import threading

client = hazelcast.HazelcastClient(
    cluster_name="hello-world",
    cluster_members=["127.0.0.1:5701", "127.0.0.1:5702", "127.0.0.1:5703"],
)

distributed_map = client.get_map("distributed-map").blocking()
distributed_map.put("key", 0)

def increment_map():
    for _ in range(10000):
        value = distributed_map.get("key")
        value += 1
```

```
distributed_map.put("key", value)

threads = []
for _ in range(3):
    t = threading.Thread(target=increment_map)
    t.start()
    threads.append(t)

for t in threads:
    t.join()

print("Count value:", distributed_map.get("key"))
client.shutdown()
```

```
[(.venv) → Task2 python3 populate.py
Count value: 13957
(.venv) → Task2 python3 populate.py
Count value: 17868
(.venv) → Task2 python3 populate.py
Count value: 17904
(.venv) → Task2 █
```

Зробіть те саме з використанням песимістичним блокування та поміряйте час

Тут також використовуємо 3 потоки, які намагаються інкрементувати значення, але цього разу використовуємо блокування, які забезпечують безпечний доступ до зміни значення. Таким чином, коли якийсь потік отримує доступ до значення він блокує доступ усім іншим, міняє та віддає доступ усім іншим, але це працює доволі довго.

```
import hazelcast
import threading
import time
```



```

client = hazelcast.HazelcastClient(
    cluster_name="hello-world",
    cluster_members=["127.0.0.1:5701", "127.0.0.1:5702", "127.0.0.1:5703"],
)

distributed_map = client.get_map("distributed-map").blocking()
distributed_map.put("key", 0)

def increment_map():
    for _ in range(10000):
        distributed_map.lock("key")
        try:
            value = distributed_map.get("key")
            value += 1
            distributed_map.put("key", value)
        finally:
            distributed_map.unlock("key")

threads = []
start_time = time.time()
for _ in range(3):
    t = threading.Thread(target=increment_map)
    t.start()
    threads.append(t)

for t in threads:
    t.join()

end_time = time.time()
print("Count value: ", distributed_map.get("key"))
print("Time taken: ", end_time - start_time)

```

```
[(.venv) → Task2 python3 populate.py  
Count value: 30000  
Time taken: 42.79241609573364  
[(.venv) → Task2 python3 populate.py  
Count value: 30000  
Time taken: 42.03502631187439  
[(.venv) → Task2 python3 populate.py  
Count value: 30000  
Time taken: 39.21838188171387  
(.venv) → Task2 █
```

Зробіть те саме з використанням оптимістичним блокуванням та поміряйте час

Цього разу використовую *replace if same*, яке забезпечують безпечний доступ до зміни значення. Якщо поточне значення не змінилося, це означає, що жоден інший потік його не оновлював між моментами перевірки та заміни, але це працює швидше ніж попередній варіант.

```
import hazelcast  
import threading  
import time  
  
client = hazelcast.HazelcastClient(  
    cluster_name="hello-world",  
    cluster_members=["127.0.0.1:5701", "127.0.0.1:5702", "127.0.0.1:5703"],  
)  
  
distributed_map = client.get_map("distributed-map").blocking()  
distributed_map.put("key", 0)  
  
def increment_map():  
    for _ in range(10000):  
        while True:  
            value = distributed_map.get("key")  
            new_value = value + 1  
            if distributed_map.replace_if_same("key", value, new_value):  
                break
```

```

threads = []
start_time = time.time()
for _ in range(3):
    t = threading.Thread(target=increment_map)
    t.start()
    threads.append(t)

for t in threads:
    t.join()

end_time = time.time()
print("Count value: ", distributed_map.get("key"))
print("Time taken: ", end_time - start_time)

```

```

[ (.venv) → Task2 python3 populate.py
Count value: 30000
Time taken: 29.06859278678894
[ (.venv) → Task2 python3 populate.py
Count value: 30000
Time taken: 27.80173397064209
[ (.venv) → Task2 python3 populate.py
Count value: 30000
Time taken: 28.459415197372437
(.venv) → Task2 █

```

Порівняйте результати кожного з запусків

Якщо порівняти попередньо використані підходи можна помітити, що :

- у варіанті без блокувань виникає втрата даних
- у песимістичному блокуванні не виникає втрати даних
- у оптимістичному блокуванні не виникає втрати даних

Втрата даних відбувається тому, що у варіанті без блокувань підхід не передбачає безпечного доступ до елемента з різних потоків, немає синхронізації, тоді як песимістичне та оптимістичне блокування забезпечують

безпечний доступ та зміну елемента. У варіанті без блокувань ми не можемо контролювати, коли та який потік отримує до доступ до елемента і через це кожного разу значення різне.

Якщо порівняти відносно часу, тоді песимістичне блокування працює довше ніж оптимістичне. Це відбувається тому, що у оптимістичному блокуванні кожен потік прямо не блокує доступ до елемента, а лише перевіряє, читає його значення, що зменшує часу виконання, тоді як у песимістичному блокуванні блокується доступ до елемента поки потік не змінить його та не поверне доступ назад.

Робота з Bounded queue

Спершу змінила максимальний розмір черги.

```
<?xml version="1.0" encoding="UTF-8"?>
<hazelcast xmlns="http://www.hazelcast.com/schema/config"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.hazelcast.com/schema/config
    http://www.hazelcast.com/schema/config/hazelcast-config-5.5.xsd">
  <network>
    <join>
      <tcp-ip enabled="true">
        <member-list>
          <member>127.0.0.1:5701</member>
          <member>127.0.0.1:5702</member>
          <member>127.0.0.1:5703</member>
        </member-list>
      </tcp-ip>
    </join>
  </network>

  <queue name="bounded-queue">
    <max-size>10</max-size>
  </queue>
```

</hazelcast>

За допомогою наступного коду створила чергу, перший потік заповнив її значенням, два наступних намагаються вичитати ці значення.

```
import hazelcast
import threading

client = hazelcast.HazelcastClient(
    cluster_name="hello-world",
    cluster_members=["127.0.0.1:5701", "127.0.0.1:5702", "127.0.0.1:5703"],
)
queue = client.get_queue("bounded-queue")

def produce():
    for i in range(100):
        queue.put(i)
    print("Producer finished.")

def consume():
    consumed_count = 0
    while consumed_count < 100:
        print("Consumed: ", queue.take().result())
        consumed_count += 1

threads = []
producer = threading.Thread(target=produce)
producer.start()
producer.join()

for _ in range(2):
    t = threading.Thread(target=consume)
    t.start()
```

```
threads.append(t)

for thread in threads:
    thread.join()

client.shutdown()
```

Яким чином будуть вичитуватись значення з черги двома клієнтами?

Після того як перший потік поклав у чергу усі елементи, два інших потоки намагаються дістати ці значення з черги. Після того як перший з двох потоків витягнув усі елементи з черги програма зависає, оскільки другий з двох потоків намагається дістати дані з черги, яка є пустою, і він очікує, коли у черзі знову з'являться дані.

```
(.venv) ➔ Task2 python3 populate.py
Producer finished.
Consumed: 0
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
Consumed: 11
Consumed: 12
Consumed: 13
Consumed: 14
Consumed: 15
Consumed: 16
Consumed: 17
```

Consumed: 18
Consumed: 19
Consumed: 20
Consumed: 21
Consumed: 22
Consumed: 23
Consumed: 24
Consumed: 25
Consumed: 26
Consumed: 27
Consumed: 28
Consumed: 29
Consumed: 30
Consumed: 31
Consumed: 32
Consumed: 33
Consumed: 34
Consumed: 35
Consumed: 36
Consumed: 37
Consumed: 38
Consumed: 39
Consumed: 40
Consumed: 41
Consumed: 42
Consumed: 43
Consumed: 44
Consumed: 45
Consumed: 46
Consumed: 47
Consumed: 48
Consumed: 49
Consumed: 50
Consumed: 51
Consumed: 52
Consumed: 53

Consumed: 54
Consumed: 55
Consumed: 56
Consumed: 57
Consumed: 58
Consumed: 59
Consumed: 60
Consumed: 61
Consumed: 62
Consumed: 63
Consumed: 64
Consumed: 65
Consumed: 66
Consumed: 67
Consumed: 68
Consumed: 69
Consumed: 70
Consumed: 71
Consumed: 72
Consumed: 73
Consumed: 74
Consumed: 75
Consumed: 76
Consumed: 77
Consumed: 78
Consumed: 79
Consumed: 80
Consumed: 81
Consumed: 82
Consumed: 83
Consumed: 84
Consumed: 85
Consumed: 86
Consumed: 87
Consumed: 88
Consumed: 89


```
Consumed: 90  
Consumed: 91  
Consumed: 92  
Consumed: 93  
Consumed: 94  
Consumed: 95  
Consumed: 96  
Consumed: 97  
Consumed: 98  
Consumed: 99
```

Перевірте яка буде поведінка на запис якщо відсутнє читання, і черга заповнена.

```
import hazelcast  
import threading  
  
client = hazelcast.HazelcastClient(  
    cluster_members=["127.0.0.1:5701", "127.0.0.1:5702", "127.0.0.1:5703"],  
)  
  
queue = client.get_queue("bounded-queue")  
  
def produce():  
    for i in range(100):  
        queue.put(i)  
        print("Produced: ", i)  
  
producer = threading.Thread(target=produce)  
producer.start()  
producer.join()
```

Запуск клієнтів.

```
hazelcast-5.5.0/bin/hz start -c /Users/mariia/hazelcast-5.5.0/config/hazelcast.xml
```

Після багатьох спроб у мене не вийшло справді обмежити чергу таким чином, щоб вона не перевищувала вказано максимального значення.